

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

The raw inputs are the acceleration in the X, Y, Z directions.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

These are spectral features derived from the inputs. By converting to frequency domain, regardless of time, specific patterns can be identified.

Five of these features are, but not limited to:

1. RMS
2. Peak 1 Freq
3. Peak 1 Height
4. Peak 2 Freq
5. Spectral Power (in different bands)

3. NN classifier outputs: State the outputs of the NN classifier.

Classifies between Nominal and Off

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

The Four features passed into the anomaly detector are spectral features derived from accelerometer data, and are:

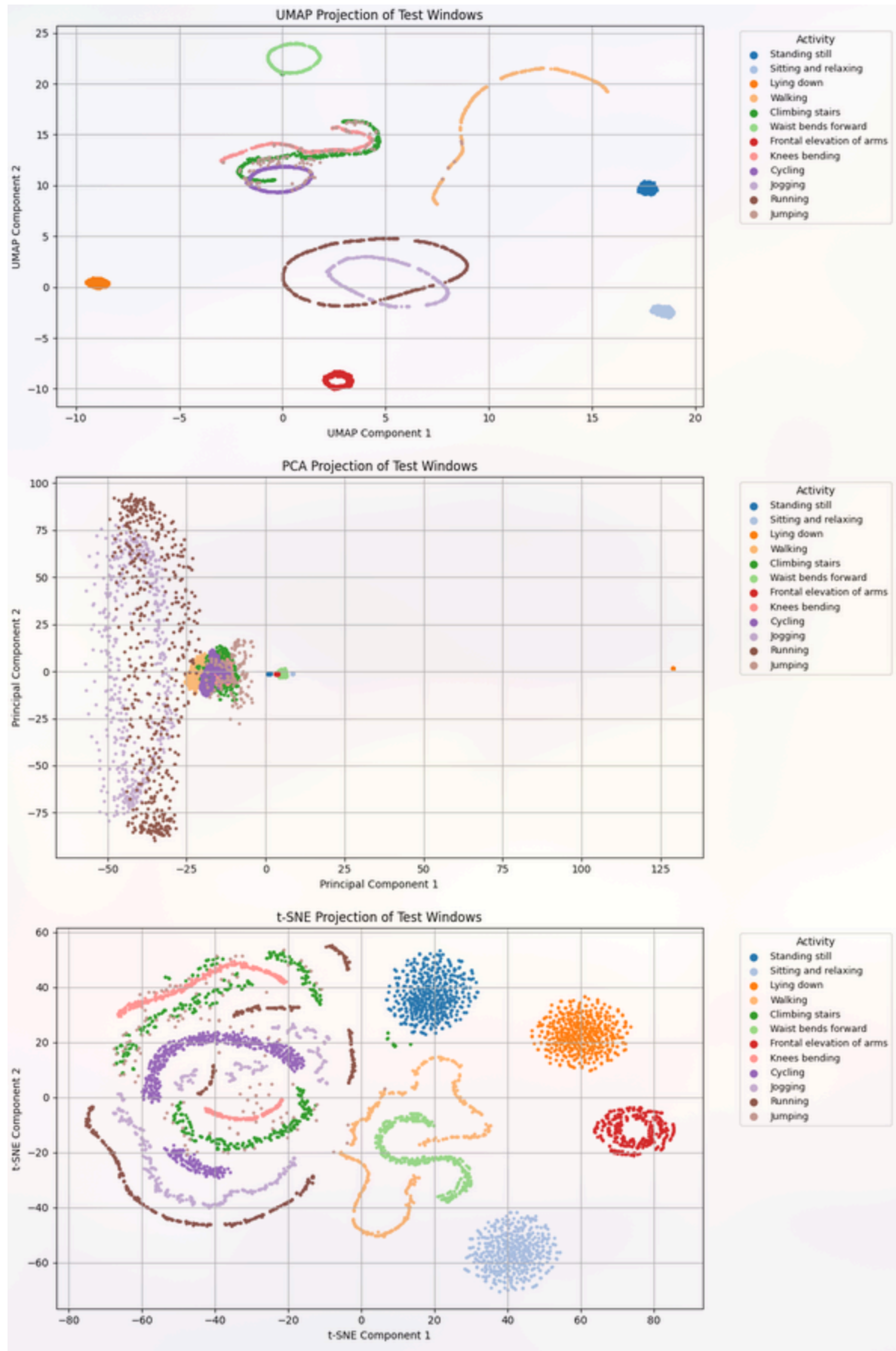
- accX RMS
- accX Peak 1 Height
- accY Spectral Power 2.0 - 5.0 Hz
- accZ Spectral Power 2.0 - 5.0 Hz

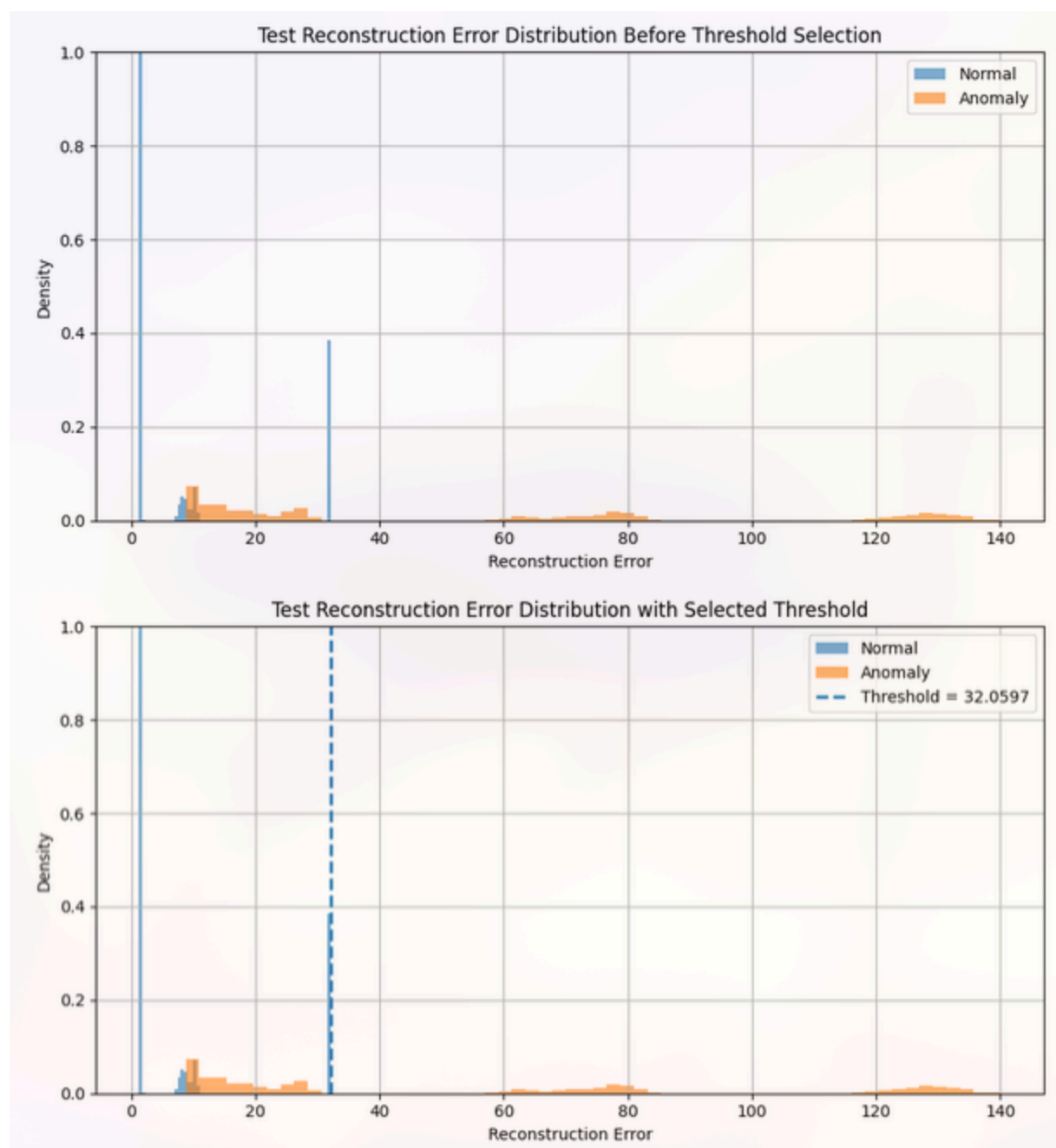
5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

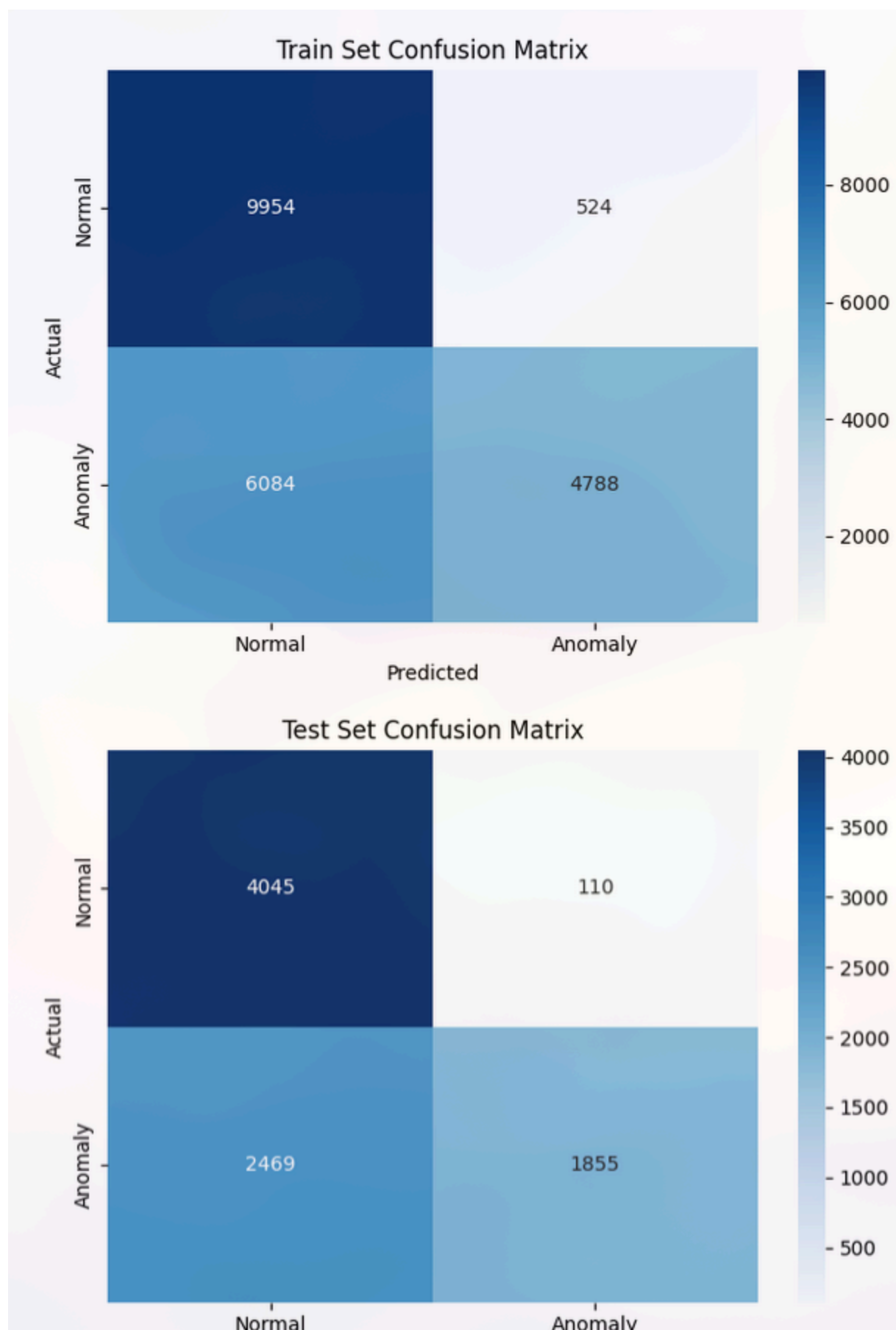
```
Inferencing settings:
    Interval: 10.00 ms.
    Frame size: 600
    Sample length: 2000 ms.
    No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 611 us, anomaly 561 us, postprocessing 49 us
#Classification predictions:
    nominal: 0.996094
    off: 0.000000
Anomaly prediction: 9.958986
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 611 us, anomaly 558 us, postprocessing 50 us
#Classification predictions:
    nominal: 0.929687
    off: 0.070312
Anomaly prediction: 57.068810
```

6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

It appears that the deployed model can sense when the motion has some kind of anomaly behind it. Classifier output should not always be trusted because it has unconfident results sometimes, or may not be able to predict novel anomalies beyond what its trained for.







```
IMU initialized.  
Allocating tensors.  
AllocateTensors successful.  
Model setup complete.  
Reconstruction error: 0.612772  
Result: Normal activity  
Reconstruction error: 0.636990  
Result: Normal activity  
Reconstruction error: 0.636963  
Result: Normal activity  
Reconstruction error: 0.636985  
Result: Normal activity  
Reconstruction error: 0.637016  
Result: Normal activity  
Reconstruction error: 0.637028  
Result: Normal activity  
Reconstruction error: 0.637023
```

What threshold did you choose for anomaly detection, and why?

- How does reconstruction error help distinguish normal and anomalous activity?

I chose the 95th percentile of training normal errors, as the threshold is derived exclusively from normal windows the autoencoder learned to reconstruct well, so it reflects the upper bound of "acceptable" error. It also avoids test set leakage by using `train_normal_errors` (not `test`), and it allows ~5% of normal windows to be false positives, while flagging anomalies whose reconstruction error lands well above this bound.

- What information from the Python notebook must be preserved when deploying the model to Arduino?

The threshold value

- Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The sketch is running the model generated by the notebook, so if they don't match it may lead to compiling errors at best, and runtime errors at worst.

- What are the main limitations of this anomaly detection method when deployed on a microcontroller?

Some main limitations of this method are that the threshold value is fixed, so changes in user behavior would require retraining, as well as the fact that the model is trained on one person's data, so it might not generalize to others.